

TRÄFF 3 AV 5

# Digital input

Modul 3: **Knapp, buzzer, logik**

NAMNGIVNA VÄRDEN

`const int` och `int`.

Ni har redan använt `const int` — för pin-nummer.

Nu behöver ni också `int` (utan `const`) — för värden som **ska kunna ändras under tiden programmet kör**: räknare, tillstånd, sensormätningar.

TUMREGEL

`const int` = sätts en gång, aldrig mer.

`int` = kan ändras.

`bool` = sant eller falskt.

```
const int knappPin = 9;    // ändras aldrig
int lastState = HIGH;      // kan ändras
bool larmPaslaget = false; // kan togglas
```

*Full förklaring av datatyper, scope och operatorer i kompendiets Bilaga A.*

# Input $\neq$ Output.

Hittills har Arduinon bara skickat ström ut (OUTPUT).

Nu läser den av ett tillstånd **in** (INPUT).

```
pinMode(knappPin, INPUT_PULLUP);  
// digitalRead(knappPin):  
//    LOW = tryckt, HIGH = släppt
```

## VARFÖR PULLUP?

En pinne som bara "lyssnar" utan koppling till plus eller minus *flyter* — läsningen blir slumpmässig.

**INPUT\_PULLUP** låter Arduinon själv dra pinnen till HIGH. När du trycker dras den till GND (LOW).

Därför: tryckt = LOW, släppt = HIGH.

FATTA BESLUT

`if` / `else` .

Läs ett värde → jämför → agera olika beroende på resultatet.

Operatorerna: `==` lika, `!=` olika, `<` mindre, `>` större.

VANLIGASTE NYBÖRJARFELET

`=` tilldelar värde.

`==` jämför.

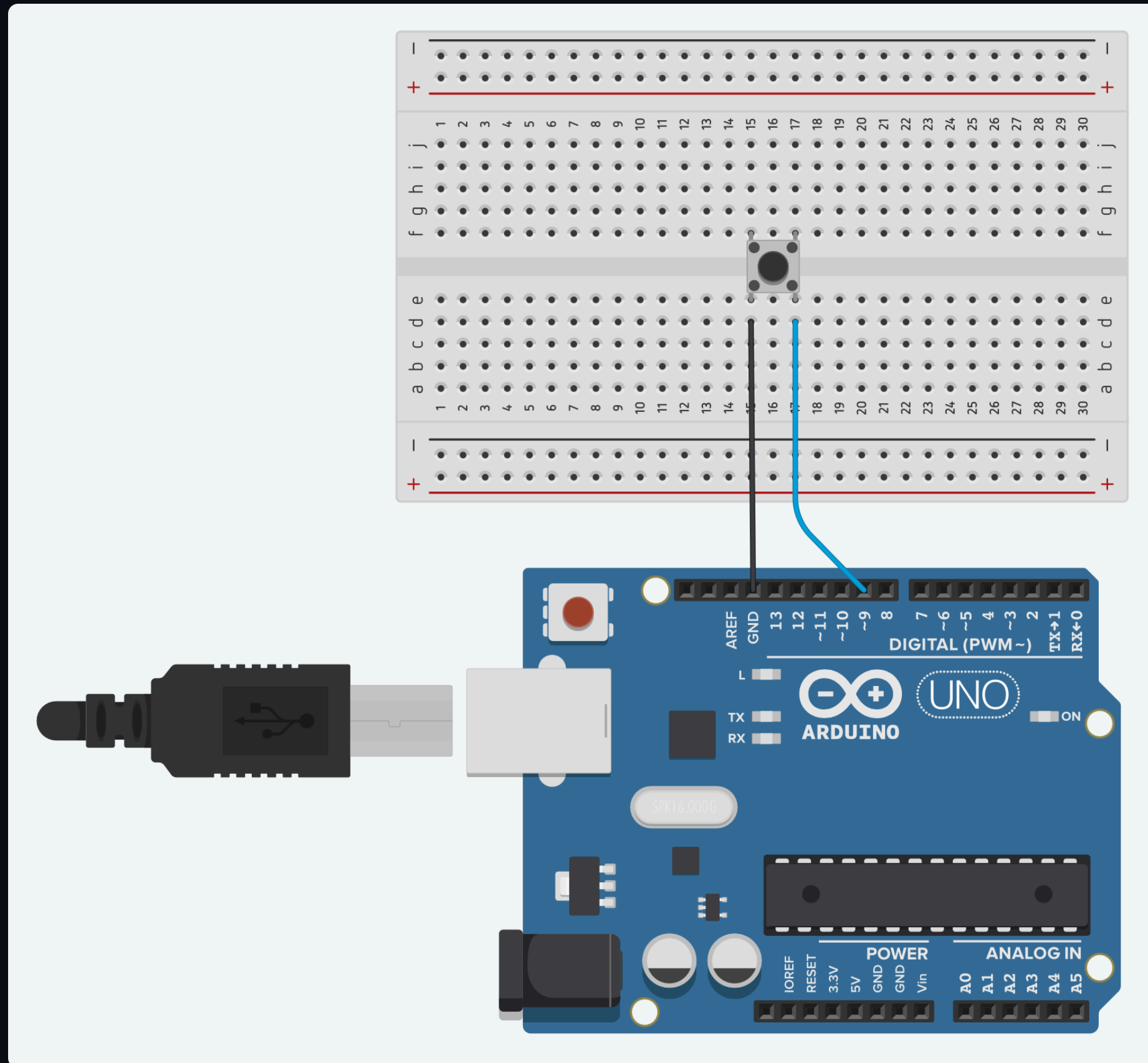
Glöm inte andra likhetstecknet.

```
if (digitalRead(knappPin) == LOW) {  
    // knappen är tryckt  
    digitalWrite(LED_BUILTIN, HIGH);  
} else {  
    // släppt  
    digitalWrite(LED_BUILTIN, LOW);  
}
```

Fler operatorer (`&&`, `||`, `!`)

och kedjade `if/else` i kompendiets Bilaga A.

# Koppla knappen.



Vi använder bara knapp A → D9 · andra benet till GND. Ingen pulldown-resistor — `INPUT_PULLUP` sköter det internt.



# Reagera på flanken.

En knapp som **hålls nere** skulle toggla hundratal gånger per sekund. Resultat: larmet blinkar mellan av/på som ett stroboskop.

Lösningen: agera inte på att knappen *är* nere. Agera på att den **just nu gick från HIGH till LOW** — flanken.

## MÖNSTRET

Spara förra värdet. Jämför med nuvarande. Om det ändrats åt rätt håll → det är en flank → agera en gång.

Inbyggda LED:en (pin 13) speglar `larmPaslaget` — så ni *ser* när larmet växlar.

```
bool larmPaslaget = false;
int lastState = HIGH;

void loop() {
    int state = digitalRead(knappPin);
    if (state == LOW && lastState == HIGH) {
        larmPaslaget = !larmPaslaget;      // flank → toggla
    }
    digitalWrite(LED_BUILTIN, larmPaslaget ? HIGH : LOW);
    lastState = state;
    delay(10); // mot studs
}
```

ACTIVE BUZZER

# En varning först.

## ❗ EXTREMT VIKTIGT

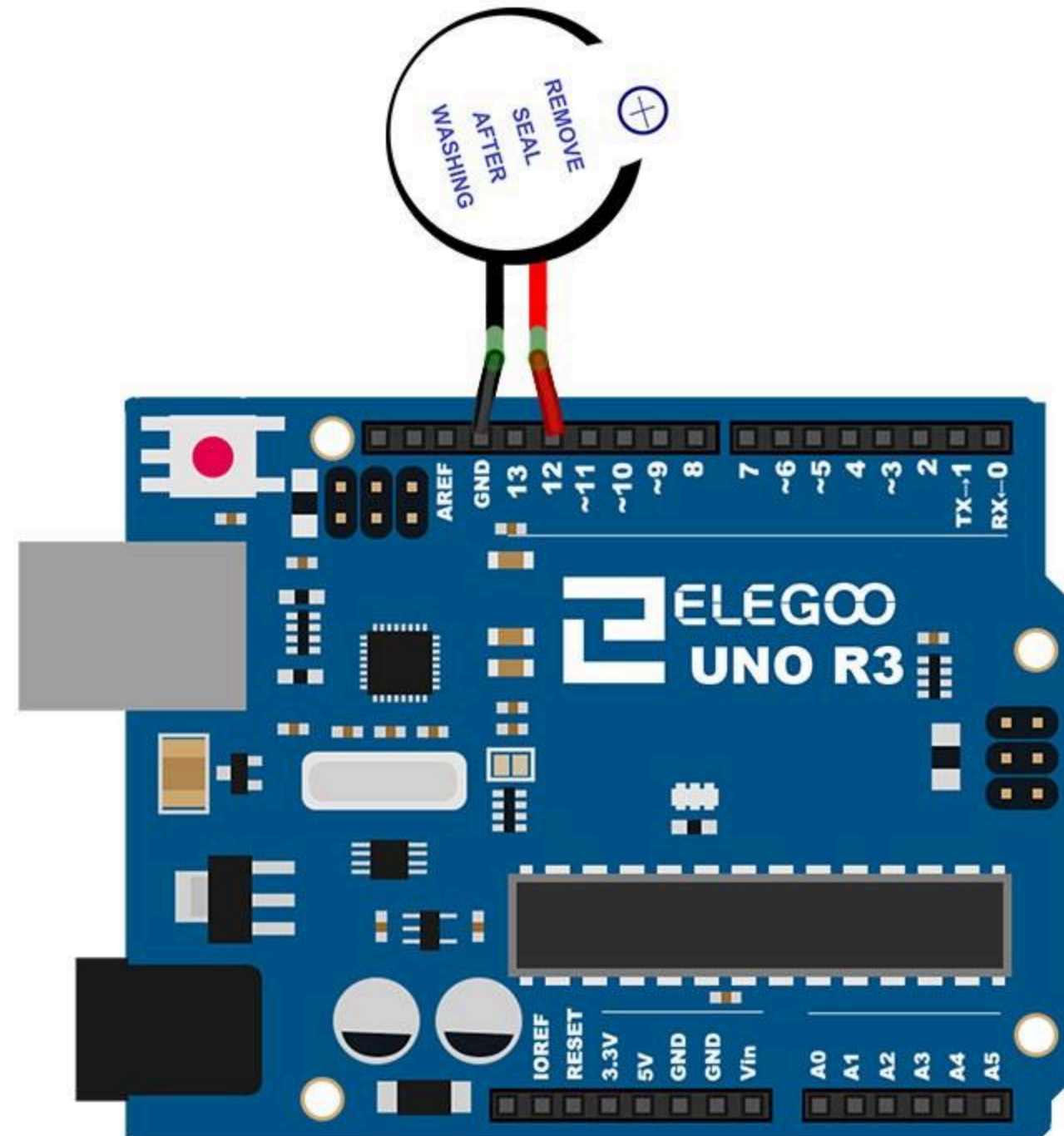
Dra absolut inte av den lilla klisterlappen  
"REMOVE SEAL AFTER WASHING"  
på er buzzer.

Utan lappen är buzzern **obehagligt högljudd**.  
Lappen är vår ljuddämpare — den stannar på.





# Koppla buzzern.



Långt ben (+) → D12 · kort ben → GND. Rakt in i Arduino-headerna. Ingen breadboard, ingen resistor.



SÄTT IHOP ALLT

# Läs knapp. Styr buzzer.

Tryck → larm på (buzzern  
tjuter).

Tryck igen → tyst.

INPUT → `flank` → OUTPUT

## Kärnan i `loop()`.

```
int state = digitalRead(knappPin);  
if (state == LOW && lastState == HIGH) {  
    larmPaslaget = !larmPaslaget;           // flank → toggla  
}  
digitalWrite(buzzerPin, larmPaslaget ? HIGH : LOW);  
lastState = state;
```

*Samma mönster som "Reagera på flanken" — LED\_BUILTIN bytt mot buzzerPin.  
Active buzzer: HIGH = pip, LOW = tyst.*

EFTER TRÄFF 3

# Arduinon lyssnar.

Ni har byggt Arduinons första **sensor** — en knapp — och gett den en **röst** — buzzern. Arduinon läser den ena — och styr den andra.

*Nästa gång: **analog sensorer**. Mörker, rörelse, mätvärden. Serial Monitor.*